EEG_API_Unicorn

Integration guide: connect an application, train the models, receive the decisions

Version of 30 October 2026

Contents

1 Overview 3
1.1 The modes 3
1.2 LSL or UDP 3
1.3 What this guide covers 3
2 Preparing the computer 5
2.1 Installing and starting the console 5
2.2 The network 5
2.3 Before each session with the headset 5
3 Connecting an application 6
3.1 Choosing a client 6
3.2 What happens 6
3.3 The LSL clients: Python, Unity, MATLAB 7
3.4 The UDP clients on a computer: Raspberry Pi, ROS 2 7
3.5 The microcontrollers: ESP32, Arduino UNO R4 WiFi 9
4 The settings of a mode 11
5 The models 12
5.1 Training a model 12
5.2 Testing a model 13
5.3 The models during a connection 13
5.4 What the model sets 13
6 Showing the stimulus: P300, ErrP, c-VEP 14
7 What the application receives 15
7.1 The decisions, mode by mode 15
7.2 The state of the engine 15
8 Writing your own client 16
8.1 The commands 16
8.2 Over LSL 16
8.3 Over UDP 17
9 Troubleshooting 19
10 Limits 20

1 Overview

EEG_API_Unicorn reads an **Unicorn Hybrid Black** EEG headset, decodes the signal and publishes the result on the local network. Your application (a game, a visualisation, a robot, LEDs…) connects to it, chooses a mode and its settings, then receives the decisions.

```
Unicorn headset ──Bluetooth──▶  the CONSOLE (on a PC)  ──network──▶  YOUR APPLICATION
(8 electrodes)                  - reads the headset                  - sends its settings
                                - trains the models                  - starts a mode
                                - decodes                            - receives the decisions
                                - publishes the decisions            - acts (game, LEDs, robot…)
```

- **The console** is the program that holds the headset. Everything is done there with buttons: check the headset, train a model, test it, connect an application.
- **Your application never touches the headset.** It talks to the console over the network, even when it runs on the same PC.
- **A decision is a neutral intent**: "target 1 is looked at", "the task LEFT is imagined". Never "move the robot forward": your application turns it into an action.

1.1 The modes

Mode	What it decodes	Model	What your application does	Over UDP
SSVEP	the target looked at, among targets that each blink at their own frequency	no	blinks the targets (on a screen or on LEDs)	yes
Motor Imagery (MI)	an imagined movement: a hand, the feet…	yes	nothing more	yes
Neuro	a mental state: load, drowsiness, engagement	no	nothing more	yes
P300	the chosen target, among targets that flash in turn	yes	flashes the targets, sends a marker at each flash	no
ErrP	the brain's reaction to a mistake of the machine	yes	sends a marker at each result shown	no
c-VEP	the target looked at, among targets that blink along a code	yes	shows the code frame by frame, sends a clock	no
Raw (`raw`)	nothing: the 8 EEG channels as they are, at 250 Hz	no	nothing more	no

SSVEP is the simplest to start with: it has no model to train and no markers to send.

1.2 LSL or UDP

Your application talks to the console in one of two ways:

	LSL	UDP
For	an application on a computer: Python, Unity, MATLAB…	a device without LSL: ESP32, Arduino, Raspberry Pi, ROS 2
Modes	all of them	SSVEP, Motor Imagery, Neuro
Engine side	the console, or the engine alone (`server.py`)	the console only
Decisions	read from the mode's stream	sent by the console to the device

LSL (*Lab Streaming Layer*) is a library that carries data streams over the local network. **UDP** is the simplest network protocol: a packet sent to an address, with no connection set up first. The commands and the replies are the same both ways.

1.3 What this guide covers

- Parts 2 to 5 are enough to connect an application with a **ready-made client**: there is only one configuration file to fill in.

- Part 6 is about the modes where your application shows the stimulus itself (P300, ErrP, c-VEP), part 7 about what a decision holds.
- Part 8 describes the protocol, to write a client for another platform.

Reference documents: `docs/control.md` (the commands, the replies, the UDP protocol), `docs/markers.md` (the markers), `docs/network.md` (the network and the firewall), and the `README.md` of each folder of `clients/`.

2 Preparing the computer

2.1 Installing and starting the console

With the installer (64-bit Windows, no Python needed):

1. Download `EEG-API-Unicorn-Setup-<version>.exe` from the Releases page of https://github.com/medkar/EEG_API_Unicorn, and run it.
2. The installer is not signed: Windows SmartScreen shows a warning the first time. Click **More info**, then **Run anyway**.
3. Choose **Install for all users** if you can: the installer then adds the Windows firewall rule (see *The network*).

Then start **EEG API Unicorn** from the Start menu.

From the sources (Python 3.12 or 3.13):

```
pip install -r requirements.txt      # once
outils\Console EEG.bat                # or a double-click in the file explorer
```

On start, the console asks two questions:

Question	Answer
The source of the session	the Unicorn headset (by its serial number), or the **test board**: a synthetic signal, to develop without a headset
The mains frequency	50 Hz in Europe, 60 Hz in America

It never falls back to the test board on its own: when the headset refuses to open, it asks again.

Language. The console opens in English. Click **“ 🌐 ”**, at the top right of the home page, to switch it to French. The data sent to your application stay in English, whatever language the console shows.

2.2 The network

- Your application and the console are on the **same local network**, in the same subnet.
- **Windows firewall**: on the console's PC, allow **EEG API Unicorn** inbound (or Python, if the console runs from the sources), public networks included. Installed for all users, the rule is already there; otherwise Windows asks the first time. For an LSL application on another PC, also allow that program (Python, Unity…) on its PC.
- Some Wi-Fi networks **isolate** devices from one another. The simplest fix is then the console PC's **mobile hotspot**: Settings ⟶ Network & internet ⟶ Mobile hotspot, at 2.4 GHz for a microcontroller. The PC is at `192.168.137.1` on it.

The details are in `docs/network.md`.

2.3 Before each session with the headset

1. **Wet the electrodes with saline**: it is the first factor of signal quality.
2. On the home page, run **“Check the headset”** (37 s). The page shows the contact of each electrode (green: good, red: bad), then measures the alpha rhythm, eyes open then closed. If it shows **“STOP HERE”**, put the electrodes back and start again: no measurement would be reliable.
3. **Keep the console open for the whole session.** When it opens again, the headset's amplifier restarts and two channels saturate for a while.

3 Connecting an application

3.1 Choosing a client

The `clients/` folder holds one ready-made client per platform. Each one does the whole connection for you. You only touch two things:

- **its configuration file**: the name of the application, the mode, its settings, the network;
- **its decision handler**: the function that receives each decision, where your code goes.

The library itself is used as it is. Copy the folder into your project, fill in the configuration, then replace the decision handler.

Folder	Platform	Transport	Configuration	Your code
`clients/python-lsl/`	Python 3 on a computer	LSL	`config.json`	`on_decision()` in `example.py`
`clients/unity/`	Unity 6	LSL	`StreamingAssets/eeg_api_config.json`	`onTargetChanged` or `DecisionReceived`
`clients/matlab/`	MATLAB R2016b or later	LSL	`config.json`	`onDecision()` in `example.m`
`clients/raspberry-pi/`	Python 3, standard library only: Raspberry Pi, Linux board	UDP	`config.json`	`on_decision()` in `example.py`
`clients/ros2/`	ROS 2	UDP	`config/params.yaml`	a node subscribed to `eeg/target`
`clients/esp32/`	ESP32 (Arduino)	UDP, Wi-Fi	`config.h`	`onDecision()` in the sketch
`clients/arduino-wifi/`	Arduino UNO R4 WiFi	UDP, Wi-Fi	`config.h`	`onDecision()` in the sketch

3.2 What happens

1. Start the console on the headset's PC.
2. Start your client. It waits to be connected; a UDP device announces itself every second.
3. In the console, on the home page: **"Connect an application"**, pick yours in the list, then **"Connect"**. For a device, **"Identify"** blinks its LEDs: you know which line is its own.
4. The client sends its settings, then starts the mode. The console now only shows the **control page**: the electrodes, the settings received, the live decoding and the last commands.
5. The mode **warms up** for 15 s, then SSVEP measures 8 s of **rest**: its first decisions come after about 23 s (40 s for Neuro). Meanwhile, the person looks at no target.
6. The decisions reach your handler. **`-1` means "no decision"**, never target 0: it is a normal, frequent value.
7. The connection ends on **"Disconnect"**, when the console closes, or when your client stops. The engine then stops the mode your client started, and the client says why the connection ended.

A **mode with a model** (Motor Imagery, and over LSL P300, ErrP, c-VEP) waits for one more step: on the control page, the person chooses the model, then clicks **"Start decoding"** (see *The models*).

One application is connected at a time. While connected, **your application owns the settings**: the console shows them and cannot change them. To change one by hand, disconnect the application first.

Without the console, with the engine alone, an LSL client of the same PC is connected at once, with no click. The engine alone listens to no UDP device. To start it, in a terminal:

Program	Command
installed	`eeg-api-unicorn-cli.exe --serveur`, in the installation folder
from the sources	`python src/core/server.py`

Both take the same options, for example `--synthetic` to open the session on the test board.

3.3 The LSL clients: Python, Unity, MATLAB

The three clients read the same keys. Here is the Python client's file, `clients/python-lsl/config.json`:

```
{
  "app_name": "my-app",
  "mode": "ssvep",
  "params": {
    "targets_on": "screen",
    "target_names": ["left", "right", "up"]
  },
  "propose_refresh_hz": 60,
  "stop_mode_on_exit": true,
  "search_timeout_s": 30
}
```

It asks for an SSVEP on a screen, with three named targets, and lets the engine choose their frequencies for a 60 Hz screen.

Key	Meaning
`app_name`	the start of the name the console lists; a random part is added, so that two copies are never mixed up
`mode`	`ssvep`, `mi`, `neuro`, `p300`, `errp` or `cvep` (Raw has no decisions: read the `EEG_API_Unicorn_raw` stream directly)
`params`	the settings of the mode (part 4); `{}` keeps the console's
`propose_refresh_hz`	SSVEP on a screen: the screen's refresh rate; the engine proposes frequencies that divide it, and they replace `freqs`. `0`: ask nothing, required for the other modes and for frequencies you give yourself. Unity: `-1` takes the rate of Unity's screen
`stop_mode_on_exit`	at the end, stop the mode the application started
`search_timeout_s`	how many seconds to look for an engine before giving up

Python and MATLAB refuse an unknown key, with the list of the known ones; Unity ignores it silently: check the spelling of the keys.

Client	Needs	Start	The name in the console
Python	Python 3 and pylsl (`pip install pylsl`)	`python example.py`	`my-app-…`
Unity	`LSL.cs` and `lsl.dll`, copied from LSL4Unity (see the folder's README)	add the **Eeg Api Client** component to a GameObject, then Play	`my-game-…`
MATLAB	liblsl-Matlab on the MATLAB path	`example`	`my-matlab-app-…`

Things to know:

- **SSVEP: your application draws the targets**, one per frequency, locked to the screen refresh. They blink from the start of the mode, during the warm-up and the rest: the engine measures its rest with the targets blinking. The frequencies kept are in the reply to the start (`params.freqs`); the Unity client exposes in `Frequencies` the ones the engine proposed (empty if you give your own `freqs`).
- **Unity: every Play creates a new application**, to be connected again in the console.
- **MATLAB** reads a one-element list of `config.json` back as a single value: give the lists of the mode at least two elements.
- Stopping the client (Ctrl+C, or Stop in Unity) stops the mode it started.

3.4 The UDP clients on a computer: Raspberry Pi, ROS 2

The Raspberry Pi client uses only the standard library of Python 3.8 or later. Its file, `clients/raspberry-pi/config.json`:

```
{
  "app_name": "my-device",
  "device_id": "",
  "device_port": 5011,
  "engine_host": "255.255.255.255",
  "engine_port": 5010,
  "mode": "ssvep",
  "params": {
    "targets_on": "physical",
    "freqs": [13, 15],
    "target_names": ["left", "right"]
  },
  "propose_refresh_hz": 0,
  "stop_mode_on_exit": true
}
```

It asks for an SSVEP on two physical targets: LEDs at 13 and 15 Hz, which the device blinks itself. The keys are those of the LSL clients, plus the network:

Key	Meaning
`device_id`	the identifier shown next to the name; empty: the end of the MAC address
`device_port`	the UDP port the device listens to, and announces (any free port)
`engine_host`	where the announcements go: `255.255.255.255` (broadcast on the network), or the IP address of the console's PC
`engine_port`	the console's UDP port: `5010`
`mode`	`ssvep`, `mi` or `neuro`: the modes served over UDP

Start it with `python3 example.py`. **"Identify"** in the console calls `on_identify()`.

ROS 2. The package `clients/ros2/eeg_api_ros/` reuses the same library and publishes every decision on topics. Its file, `config/params.yaml`, sets its two nodes:

```
# The settings of the two nodes. This is the only file to edit.

eeg_api_bridge:
  ros__parameters:
    app_name: "my-robot"                # the name the console lists under "Connect an application"
    device_id: ""                       # empty: the end of this computer's MAC address
    device_port: 5011                   # the UDP port this node listens to, and announces
    engine_host: "255.255.255.255"      # where the hellos go: broadcast, or the console's IP address
    engine_port: 5010                   # the UDP port of the console
    mode: "ssvep"                       # ssvep, mi or neuro (the modes served over UDP)
    # The settings of the mode, as JSON (keys and ranges: docs/control.md). Here: two physical
    # targets (LEDs) blinking at 13 and 15 Hz.
    settings: '{"targets_on": "physical", "freqs": [13, 15], "target_names": ["forward", "turn"]}'
    propose_refresh_hz: 0.0             # > 0: SSVEP on a screen, ask the engine for frequencies
    stop_mode_on_exit: true

intent_to_twist:
  ros__parameters:
    linear_x: [0.1, 0.0]                # m/s for target 0, target 1...
    angular_z: [0.0, 0.5]               # rad/s for target 0, target 1...
    timeout_s: 1.0                      # no decision for this long: zero velocity
    cmd_vel_topic: "cmd_vel"
```

The keys of the bridge (`eeg_api_bridge`) are those of the Raspberry Pi, with one difference: the settings of the mode are written as JSON text, in `settings`, since a ROS parameter cannot hold a free-form object.

Topic	Type	Content
`eeg/target`	`std_msgs/Int32`	the decoded target (SSVEP) or task (MI); `-1`: no decision
`eeg/confidence`	`std_msgs/Float32`	the score of the decision
`eeg/decision`	`std_msgs/String`	the whole decision, as JSON
`eeg/state`	`std_msgs/String`	`announcing`, `connected`, `decoding`, `disconnected` or `failed`

Your node subscribes to `eeg/target`. The example node `intent_to_twist` turns it into a velocity: the velocity of the decoded target, or zero on `-1` and when no decision came for `timeout_s`. Copy `eeg_api_ros/` into the `src/` folder of a colcon workspace, go to its root, then:

```
colcon build --packages-select eeg_api_ros        # again after every change to params.yaml
source install/setup.bash
ros2 launch eeg_api_ros eeg_api.launch.py         # the bridge and intent_to_twist
ros2 launch eeg_api_ros eeg_api.launch.py with_example:=false    # the bridge alone
```

3.5 The microcontrollers: ESP32, Arduino UNO R4 WiFi

The ESP32 connects to the console over Wi-Fi, with no computer in between. For SSVEP, its LEDs **are** the targets: they blink in the board's hardware, at frequencies that neither the Wi-Fi nor the program can shift. Its file, `clients/esp32/esp32_client/config.h`:

```
// The settings of esp32_client: the only file to edit. Everything else works as it is.
// ⚠ This file holds your Wi-Fi password: do not publish it.

// --- The network -------------------------------------------------------------------------

// The Wi-Fi network the console's computer is on too: that computer's mobile hotspot, or the
// Wi-Fi of a router. At 2.4 GHz: the ESP32 does not see 5 GHz.
#define WIFI_SSID "network-name"
#define WIFI_PASS "password"

// Optional, a second network: the ESP32 takes the one it sees, the strongest if it sees both.
// #define WIFI_SSID_2 "network-name-2"
// #define WIFI_PASS_2 "password-2"

// Where the announcements go. "" = broadcast on the local network, which is usually enough.
// Otherwise the IP address of the console's computer (useful when the console never lists the
// device: a computer on several networks, a VPN). On the Windows mobile hotspot, the computer is
// at 192.168.137.1.
#define CONSOLE_IP ""

// --- The application ---------------------------------------------------------------------

// The name the console lists under "Connect an application", and the identifier shown next to
// it ("" = the end of the board's MAC address).
#define APP_NAME "esp32-app"
#define DEVICE_ID ""

// The UDP port this device listens to (any free port), and the console's (5010).
#define DEVICE_PORT 5011
#define ENGINE_PORT 5010

// The mode: "ssvep" (the LEDs ARE the targets: each one blinks at its frequency), "mi" (the LED
// of the decoded task lights up) or "neuro" (the values go to the serial monitor).
#define MODE "ssvep"

// One LED per target (SSVEP), or per task (MI, in the order of the model's tasks). Each LED has a
// resistor in series: (3.3 V - the LED's forward voltage) / the current, under about 20 mA per
// pin; 33 ohm suits a white or blue LED (about 3 V), about 100 ohm a red one.
#define TARGET_PINS {25, 26}
```

```
// SSVEP: the frequency (Hz) and the name of each target, in the order of TARGET_PINS. On LEDs any
// frequency works (no screen refresh rate to divide), at least 0.67 Hz apart.
#define TARGET_FREQS {13.0, 15.0}
#define TARGET_NAMES {"left", "right"}

// Other settings of the mode, a JSON object (docs/control.md, "Settings"). For SSVEP, targets_on,
// freqs and target_names are added from the lines above. Example: "{\"band_low\": 6}".
#define EXTRA_SETTINGS "{}"

// SSVEP: the chosen target stays steady for HOLD_MS, then decisions are ignored for REARM_MS more
// (the decoder's window still contains the pause).
#define HOLD_MS 3000
#define REARM_MS 1500

// The board's LED: on while a console is connected, blinking after a refusal. -1 = none.
#define STATUS_LED 2
```

⚠️ **This file holds the Wi-Fi password: do not publish it.** The Wi-Fi must be at 2.4 GHz: the ESP32 does not see 5 GHz.

What the board does, by `MODE`:

`MODE`	What the board does
`"ssvep"`	each LED blinks at its frequency; the decoded target stays steady for `HOLD_MS`, then all blink again
`"mi"`	the LED of the decoded task lights up, the others go off; none on `-1`
`"neuro"`	the values go to the serial monitor

At most 4 pins in `TARGET_PINS`: the board has four hardware timers, one per target. `TARGET_FREQS` and `TARGET_NAMES` have as many elements as `TARGET_PINS`, even in `"mi"`, where the frequencies are not used: otherwise, the sketch does not compile.

The board's LED (`STATUS_LED`) is on while a console is connected, and blinks after a refusal. The serial monitor (115200 bauds) says what the board does, and the reason for a refusal.

Wiring. One LED per target, from the pin of `TARGET_PINS` to GND, long leg on the pin side, with a resistor in series: (3.3 V − the LED's voltage) / the current, under about 20 mA per pin. 33 Ω suits a white or blue LED (about 3 V), about 100 Ω a red one.

Build and upload. You need the ESP32 boards support and the ArduinoJson 7 library. In the Arduino IDE: install "esp32" (Espressif) in the Boards Manager and "ArduinoJson" in the Library Manager, open `esp32_client.ino`, pick the board ("DOIT ESP32 DEVKIT V1" for an ESP32 DevKit V1), then Upload. With arduino-cli:

```
arduino-cli core install esp32:esp32
arduino-cli lib install ArduinoJson
arduino-cli compile --fqbn esp32:esp32:esp32doit-devkit-v1 esp32_client
arduino-cli upload --fqbn esp32:esp32:esp32doit-devkit-v1 -p COM5 esp32_client
```

(`COM5` is an example: `arduino-cli board list` gives the board's port.)

Arduino UNO R4 WiFi. The client `clients/arduino-wifi/` uses the same library, with an almost identical `config.h`: `MODE` is `"mi"` or `"neuro"`, one LED per task in `TASK_PINS` (about 220 Ω at 5 V), and the settings of the mode in `SETTINGS`. For an SSVEP on LEDs, take an ESP32: only it blinks the targets in hardware. Board: "Arduino UNO R4 WiFi", support "Arduino UNO R4 Boards" (`arduino:renesas_uno:unor4wifi`).

4 The settings of a mode

The client sends the settings before it starts the mode. They are written in `params` (LSL clients and Raspberry Pi), `settings` (ROS 2), `EXTRA_SETTINGS` (ESP32) or `SETTINGS` (UNO R4).

Mode	Setting	Meaning
`ssvep`	`targets_on`	`"screen"` (targets on a screen) or `"physical"` (LEDs)
	`freqs`	one frequency per target, in Hz (2 to 10 targets)
	`target_names`	one name per frequency, in the same order (optional)
	`refresh_hz`	the refresh rate of the stimulus screen (screen only)
	`alpha_hz`	the person's alpha peak, measured by "Check the headset"
`mi`, `p300`, `errp`, `cvep`	`model`	the model's file name: with the console, a mere suggestion
`p300`, `errp`, `cvep`	`stream_in`	the name of the marker stream your application publishes

The full list, with the range of each setting, is in `docs/control.md`.

- **The engine is the only judge.** A refused setting comes with its reason, in English, and the client shows it. Do not write the engine's rules into your application: read its refusals.
- **SSVEP on a screen**: a target can only blink at a frequency that divides the refresh rate (at 60 Hz: 30, 20, 15, 12, 10, 8.571…), and the engine refuses the others. Do not write the frequencies by hand: give `propose_refresh_hz`, and the client asks the engine for a set suited to the screen and to the alpha peak.
- **SSVEP on LEDs** (`"targets_on": "physical"`): any frequency of the filter band will do, as long as two targets are at least 0.67 Hz apart.
- A setting changed while the mode runs often restarts the warm-up and the rest.

5 The models

Four modes decode nothing without a model: Motor Imagery, P300, ErrP and c-VEP. A model is learned on **the person** who wears the headset, with **this** headset, during **the same session** if possible. Another person's model raises no error: it gives plausible answers, but wrong ones.

Mode	What the model learns	Training time
Motor Imagery	the imagined tasks (2 to 5, LEFT and RIGHT by default)	shown by the page, from the tasks and the trials
P300	the response to the flashes of the chosen target (3 to 10 targets, 6 by default)	about 2.2 min at 6 targets
ErrP	the reaction to a mistake of the machine	about 5.7 min
c-VEP	the response to the blinking code (2 to 10 targets, 6 by default)	about 3.1 min at 6 targets and 60 Hz

SSVEP and Neuro have no model: they measure a rest at each start.

5.1 Training a model

Training is done **in the console only**: no command lets an application train a model.

1. Pass **"Check the headset"** first.
2. Open the mode's page: its tile, then **"Open"**. The page has numbered steps: **1. Set up**, **2. Train**, **3. Test**. Click **"Train"**.
3. Choose what the model will learn, **before** you begin: it will keep these choices.

Mode	To choose
Motor Imagery	the number of tasks and their names (LEFT, RIGHT, or FEET, HANDS…), the lead-in (1 to 3 s left to start imagining before the recording), the number of trials per task
P300, c-VEP	the number of targets and their names; for c-VEP, the refresh rate of the stimulus screen (60 Hz by default)
all four	the filter band: the default value is usually right

   ⚠️ Without a rest task, a Motor Imagery model **always** names a task: only its probability threshold (`prob_min`) silences it, by publishing `-1` when it is not sure enough. If your application must know when the person does nothing, add a REST task.
4. Click **"Begin"**. The console first checks the link with the headset and shows the contact of each electrode. If the signal of a single one of the eight electrodes leaves [0.5; 500] µV, or if the reference is loose, it refuses and says why: fix it, then start again. Otherwise, click **"Begin training"**.
5. Follow the instructions. For Motor Imagery, the console shows them itself. For P300, ErrP and c-VEP, it opens a **stimulus window** on the main screen (for c-VEP, a black screen for one second at the start and at the end is normal).
6. At the end, the console shows the **result**: a level (green, orange or red), a figure, and the chance level to compare it with. Two buttons:
   - **"Save model"**: the model is written to the models folder, under a timestamped name (for example `p300_model_20261029_101500_6targets.joblib`);
   - **"Redo"**: nothing is kept.

The models folder is `Documents\EEG API Unicorn\modeles\` for the installed program, and `data\` at the root of the repository from the sources. **Nothing is written there before the "Save model" click.** c-VEP saves two models per session, one per algorithm (eCCA and rCCA), with comparable performance.

The order of a session is therefore: **train, save, test, then connect**. While a result waits for "Save model" or "Redo", the console refuses to connect an application; while connected, no training is possible.

5.2 Testing a model

A saved model is not necessarily a good one. The **“Test”** block of the mode’s page checks it:

Button	What it does
“Guided test”	plays the training protocol again, but the engine **decides** instead of learning, and gives a score compared with chance
“Free test” (c-VEP, SSVEP, P300, Motor Imagery)	runs the mode and circles what it decodes live, with no score

For SSVEP, which has no model, “Guided test” is a guided session; it is disabled with physical targets, since it points at discs on the screen.

⚠️ Two tests often give a red result even when everything works: the Motor Imagery one, which would need many more trials to rule out chance, and the c-VEP one, whose green is demanding at the default length. Do not conclude on a single test.

5.3 The models during a connection

When an application is connected, **the person at the console chooses the model**, never the application:

1. Your client starts a mode with a model. The engine replies that the mode **waits** (`"deferred": true`).
2. On the control page, the person ticks a model, by its file name.
3. They click **“Start decoding”**. The mode starts, and the decisions arrive.

- The `model` your application gives is only a **suggestion**: the console ticks it in advance if it exists, and never applies it by itself.
- **If no trained model exists** for this mode on the console’s PC, the start is refused: disconnect the application, train a model, then connect it again.
- If the person changes the model (**“Restart with this model”**), the stream of decisions is recreated; the ready-made clients open it again by themselves.

5.4 What the model sets

The model **carries** its parameters: the number of targets or tasks, their names, their order, the refresh rate (c-VEP), the filter band. Do not hard-code them in your application:

- **over LSL**, they are in the description of the stream of decisions, under the `decoding` element: `n_targets` (P300, c-VEP), `frequencies_hz` (SSVEP), `classes` (the MI tasks), `refresh` (c-VEP) and `target_names` when names are set. Each value is a text, and a list is joined with commas;
- **over UDP**, a device does not receive the names of the Motor Imagery tasks, only their number. Their order is the one chosen at training, shown on the control page: wire the LEDs of `TASK_PINS` (or `TARGET_PINS`) in that order.

6 Showing the stimulus: P300, ErrP, c-VEP

For these three modes, **your application shows the stimulus** and tells the engine, at the exact instant, what it has just shown. It does so through an LSL stream of **markers**, named `EEG_API_Unicorn_stim` by default. Without markers, the mode decodes nothing. The ready-made clients handle the connection; the markers are up to your application. They do not go over UDP, since they are timestamped in the LSL clock.

Mode	Marker	When to send it
P300	`{"mode": "p300", "event": "flash",` , then `"target": 3}` `{"mode": "p300", "event": "round_end"}`	at each flash, then at the end of each round (each target flashes 2 to 8 times per round)
ErrP	`{"mode": "errp", "event": "feedback"}`	at each result shown, right or wrong
c-VEP	`{"mode": "cvep", "event": "cycle", "refresh": 60.0}`	each time the code starts again from its first frame, about once a second

Three rules make the decoding:

1. **Timestamp each marker right AFTER the frame is shown** (after the `flip`), never before. Forty milliseconds early is enough for the engine to decode noise, without any error.
2. **Show the stimulus the model learned**: for P300, targets numbered like the model's (`target` from 0 to `n_targets - 1`); for c-VEP, the same code, at the same lags and the same refresh rate (those of `src/core/cvep_code.py`).
3. **Never send the answer** while decoding: no `error` field on the ErrP marker, no target on the c-VEP one.

Rules of each mode:

- **P300**: leave about 2.5 s between a `round_end` and the next flash, for the gaze to move to another target. Space the flashes by 150 ms, as in training. Always send the `round_end`, even when the round is interrupted: without it, the round is dropped after 10 s.
- **P300 and ErrP**: markers sent during the warm-up (15 s; 23 s for ErrP, rest included) are dropped. Begin when the phase is `decoding`.
- **c-VEP**: `refresh` is a number, equal to the model's within 1 Hz; otherwise the markers are refused and the mode publishes `-1` forever. Send a marker at each cycle: after 3 cycles without one, the mode stops decoding. Blink from the start of the mode, warm-up included: c-VEP keeps those markers.

If several senders may publish on the same network, give yours a unique name (for example `EEG_API_Unicorn_stim_my-game`) and put it in the `stream_in` setting, **before** starting the mode. If another marker mode already runs on another stream, stop it first: the mode you start would listen to that one.

The full protocol is in `docs/markers.md`. The windows of `src/stimulus/` (`p300.py`, `errp.py`, `cvep.py`) are its reference implementations.

7 What the application receives

7.1 The decisions, mode by mode

A decision holds the channels of the mode's stream, by name:

Mode	Channels	Rate
SSVEP	`target_index`, `freq_hz`, `confidence`, one `score_<f>Hz` per frequency	about 5 per second
Motor Imagery	`intent_index`, `confidence`, `p_0` … `p_<n−1>` (one probability per task)	about 5 per second
Neuro	`charge`, `somnolence`, `engagement`, `artifact`	about 5 per second
P300	`target_index`, `confidence`, `n_flashes`, `score_0` …	one per round
ErrP	`error`, `score`, `threshold`, `artifact`	one per result shown
c-VEP	`target_index`, `confidence`, `score_0` …, `corr_min`, `margin`	about 5 per second

- **`-1` on `target_index` or `intent_index` means "no decision"**, never target 0. A mode that is not sure publishes `-1`: it is often the most frequent value. Treat it as "do nothing".
- **ErrP**: `error` is `1` (error), `0` (no error) or `-1` (no verdict, which is not a `0`).
- **A silent stream** means something else: the mode warms up, measures its rest or is stopped; or P300 and ErrP receive no markers.
- **Over UDP**, every channel arrives as a number: `target_index` is `1.0`, and `-1.0` means "no decision".
- The **Neuro** values are relative to the rest measured at the start, for this person and this session: they compare neither across people nor across sessions. They can be `-1` without meaning "no decision".
- **Each mode has its own scale** (`decision_scale`, in the stream's description): SSVEP decides on a `z`, MI on a probability, P300 and ErrP on a `logodds`, c-VEP on a correlation. A threshold of one mode means nothing for another.
- **P300**: `confidence` is a log-odds, mostly negative. Do not filter on `confidence > 0`: you would discard every right answer. Read `target_index`.
- During a **test** run in the console, the mode's stream carries the test's decisions: your application receives them too.

To see the streams as an application receives them, open **"What your app sees"**, at the bottom of the console's home page.

7.2 The state of the engine

The `EEG_API_Unicorn_status` stream, published again every 2 s, describes the engine:

Field	Content
`modes`	the list of the running modes, for example `["raw", "ssvep"]`
`phase`	`warmup`, `baseline` (the rest: look at no target), `decoding`, `calibrating` (a training) or `streaming`; with several modes, that of the least advanced
`instruction`	the instruction during the warm-up and the rest
`marqueurs`	what the engine hears of the markers: `connecte` (false: it hears no marker stream), `perdus`, `futurs`, `illisibles`, `inlet_erreurs`. The first place to look when a P300, ErrP or c-VEP decodes nothing

Over UDP, the engine pushes a shorter `status`, limited to the modes the device asked for. ⚠️ There, `modes` is a dictionary mode → phase (`{"ssvep": "decoding"}`), and `phase` can also be `idle` or `deferred` (a mode with a model waits for its model to be chosen).

8 Writing your own client

This part is for writing a client on a platform that has none. The two libraries `clients/python-lsl/eeg_api_lsl.py` (LSL) and `clients/raspberry-pi/eeg_api_udp.py` (UDP) are its reference implementations, and `docs/control.md` is the full contract.

8.1 The commands

There are four commands, and no other. They are the same over LSL and over UDP.

Command	Effect
`set_params`	changes settings. On a stopped mode, they are **checked, then kept** for the next start (`"deferred": true`); on a running mode, they apply
`start_mode`	starts the mode with its kept settings; already running: `"already_running": true`
`stop_mode`	stops the mode; already stopped: `"already_stopped": true`
`propose_params`	asks the engine to **compute** a setting without applying it: the SSVEP frequencies for a given screen

A command is a JSON object, sent as one text, with a unique **`ref`**:

```
{"cmd": "set_params", "mode": "ssvep", "params": {"targets_on": "physical",
 "freqs": [13, 15], "target_names": ["left", "right"]}, "ref": "my-game-1"}
```

The reply copies this `ref`, and lists every setting of the mode (shortened here):

```
{"ref": "my-game-1", "cmd": "set_params", "mode": "ssvep", "accepted": true,
 "deferred": true, "warning": null, "params": {"targets_on": "physical", "freqs": [13.0, 15.0]}}
```

- **The normal order: set up, then start.** Send `set_params`, wait until it is accepted, then send `start_mode`.
- **A command is safe to send again.** With no reply after a few seconds, the command or its reply was lost: send it again, with the same `ref`.
- **A refusal** carries `"accepted": false` and a `reason` in English, which says the cause and often the fix. Show it.
- **An accepted start can still fail** a moment later (a model file that cannot be read): a second message arrives, with `"event": "failed"` and the `ref` of the `start_mode`.
- **SSVEP on a screen**: send `propose_params` with `{"refresh_hz": 60}`. The reply gives `key` (`freqs`) and `value` (the frequencies). Apply them together with `set_params`: `{"refresh_hz": 60, "freqs": <value>}`. A `refresh_hz` sent alone is refused as soon as the kept frequencies do not divide it.
- Beyond about 20 commands per second, the next ones are refused.

8.2 Over LSL

Stream	Direction	Created by	One sample is
`EEG_API_Unicorn_control`	application → engine	**your application**	one command, as JSON text
`EEG_API_Unicorn_replies`	engine → application	the engine	one reply, as JSON text
`EEG_API_Unicorn_decoded_<mode>`	engine → application	the engine	one decision

The first two are of type `Markers`, one channel, format `string`, irregular rate.

In this order:

1. **Open every `EEG_API_Unicorn_replies` stream of the network**, before any command: a reply published before they are open is lost.
2. **Create the command stream.** Its `source_id` is the name the console shows in its list: make it explicit, in plain ASCII, and unique if several copies may run. Without a readable name, the disconnection message could not point at you.
3. **Wait until the engine reads it** (`wait_for_consumers`): a command sent before is lost. With the console, this wait lasts until the click on “Connect”: do not cut it short, and tell the user what is awaited.

4. **Send the commands**, each with a unique `ref`: the replies stream is shared by every application of the network, and the `ref` tells yours apart.
5. **Open the stream of decisions of THAT engine.** Every engine publishes under the same name; they differ by their `source_id`, `EEG_API_Unicorn_decoded_<mode>@<engine id>`. The id is that of the replies stream that answered you.
6. **Read the decisions.** If the engine recreates the stream (new settings, new model), open it again.
7. **At the end**, send `stop_mode`, then close the command stream: the connection ends with it.

The disconnection comes as a message on the replies stream, which answers no command:

```
{"ref": null, "event": "disconnected", "cause": "console", "app": "my-game",
 "stopped": ["ssvep"], "reason": "Disconnected in the console: ..."}
```

Field	Meaning
`cause`	`"console"`: the person clicked “Disconnect”; `"engine_stopped"`: the engine stopped
`app`	the `source_id` of the command stream concerned: keep only the message that carries yours
`stopped`	the modes the engine stopped
`reason`	the same thing, in English, for people: your program must not depend on its wording

To go on after a disconnection: close the command stream, create a new one, wait to be connected again, then send the settings and the `start_mode` again. ⚠️ Do not rely on `have_consumers()` to notice that the engine left: LSL only notices at the next send.

8.3 Over UDP

The console listens to UDP on **port 5010**. Six rules:

1. **One datagram = one JSON object, in UTF-8, carrying `"v": 1`**, 8 KB at most. Without `"v": 1`, it is ignored with no reply.
2. **Send everything from the announced port**, with a single socket bound to that port: the engine recognises the device by the address its datagrams come from. A datagram from another port is ignored with no reply.
3. Once connected, **reply to the address the `connect` came from**: it is the engine’s.
4. Every message pushed by the engine carries an increasing **`seq`**: ignore one whose `seq` is not greater than the last one received, and start again from 0 at each `connect`.
5. **UDP can lose a datagram**: send again a command left 2 s without a reply. Read with an 8 KB buffer: the replies are often longer than 255 bytes.
6. **Once connected, send `{"v": 1, "alive": true}` every second.** After 5 s with nothing valid from the device, the engine considers it gone and stops what it started.

Step	Sender	Datagram
Announce, every second, until connected	device → `255.255.255.255:5010`, or the console’s IP	`{"v": 1, "hello": "EEG_API_Unicorn", "app": "my-game", "id": "AB:CD", "port": 5011}`
Identify (a click, optional)	engine → device	`{"v": 1, "identify": true}`: blink an LED
Connect (a click)	engine → device	`{"v": 1, "connect": "<engine>", "host": "<PC name>"}`
	device → engine	`{"v": 1, "connected": true}`; already connected elsewhere: `{"v": 1, "connected": false, "busy": "<other PC>"}`
Commands	device → engine	`{"v": 1, "cmd": "start_mode", "mode": "ssvep", "ref": 1}`
	engine → device	the same reply as over LSL, plus `"v"` and `"seq"`
While connected	engine → device	`{"v": 1, "seq": n, "status": {...}}` at each change and every second; `{"v": 1, "seq": n, "decision": {...}}` at each decision
	both	`{"v": 1, "alive": true}` every second
End	device → engine	`{"v": 1, "bye": true}`
	engine → device	`{"v": 1, "seq": n, "event": "disconnected", ...}`

- The word `"EEG_API_Unicorn"` is written exactly like this, and `port` is an integer.
- The engine sends `connect` **only once** and waits 3 s: with no reply, the console says the device does not answer, and the person has to click again.
- After a `disconnected`, as after 5 s with nothing from the engine, the device considers itself disconnected and announces itself again.
- The engine's `identify`, `connect` and `alive` messages carry no `seq`.
- On an ESP32, a datagram longer than about 1470 bytes never arrives; the messages of the modes served over UDP are much shorter.

9 Troubleshooting

Symptom	Likely cause
the application is not in the console's list	it is not on the same network, or a firewall blocks it (allow the program, public networks included); some Wi-Fi networks isolate devices: use the mobile hotspot of the console's PC
a UDP device is not listed	its broadcast does not reach the console (a PC on several networks, a VPN): put the console's IP address in `engine_host` or `CONSOLE_IP`; or the engine alone is running, not the console
the console says the device does not answer	it did not answer the `connect` within 3 s, or it answered from another port than the one it announced
the console says the device gives no more news	nothing valid received for 5 s: its `alive` lacks `"v": 1`, or leaves from another port than the one it announced
the console says the device is already connected	it answered `"busy"`: another console holds it; disconnect it there first
the client waits to be connected, endlessly	nobody clicked "Connect"; with the engine alone, the application does not run on the same PC
`"accepted": false`	the `reason` says why: a setting out of range, an unknown mode, no trained model…
the mode waits (`"deferred": true`)	a mode with a model: the person chooses the model, then clicks "Start decoding"
a started mode publishes nothing	it warms up (15 s) then measures its rest (about 23 s in all for SSVEP, 40 s for Neuro); P300 and ErrP decode nothing without markers
decisions, but random ones	the contact of the electrodes (run "Check the headset" again), the model of another person or session, a stimulus that differs from the model's, or SSVEP frequencies that differ from those blinking
the mode stops when the application restarts	it recreated its command stream: it is a new application, to be connected again
the console refuses to connect the application	a training result waits for "Save model" or "Redo", or a test is running: finish it first
your own client gets no reply	the command left before the engine read the stream, or the replies were opened too late: send it again
you do not know what flows on the network	open "What your app sees" in the console: every stream and its values, as an application receives them

10 Limits

- **One application** is connected to a console at a time.
- **Local network only**: LSL discovery and UDP broadcast do not cross routers. The exchanges are neither encrypted nor authenticated: only the click on “Connect” authorises an application.
- **Over UDP**, only SSVEP, Motor Imagery and Neuro are served; P300, ErrP, c-VEP and Raw are refused, with their reason. Motor Imagery does not send the names of the tasks there.
- **The decoding performance** depends on the person, the contact of the electrodes and the session: measure it with “Test” before relying on it.